

Optical Network Simulator

Fresher Ramp-Up Guide

A 12-page introduction to the product, its UI, and the engineering behind it

VCTI · Internal · May 2026 · Version 1.0

1. Read this first

You have just joined the team building the **Optical Network Simulator (ONS)** at VCTI. This document is your fast on-ramp. It is intentionally short: 12 pages, designed to be read end-to-end in under thirty minutes. By the end you will know *what* ONS is, *why* it exists, *how* it is built, and *where* to look next.

Audience. New engineers, QA hires, interns, integrators. **Pre-requisites.** None. We assume zero prior optical-networking knowledge. **Stack.** Python 3.12 + FastAPI on the backend, React 18 + TypeScript on the frontend, PostgreSQL 16, Redis 7.

What you will find in this document

- 1. Read this first — page 2
- 2. What ONS is, in one page — page 3
- 3. Optical networking in plain English — page 4
- 4. The five tabs (product tour with screenshots) — pages 5–8
- 5. System architecture at a glance — page 9
- 6. Technology stack, by layer — page 10
- 7. How to run it on your laptop — page 11
- 8. Where to go next — page 12

The 60-second elevator pitch

Optical transport networks — the glass-fibre roads that carry every byte of internet traffic between cities — are *expensive to build, slow to lab up, and dangerous to experiment on in production*. ONS is a software product that models such a network end-to-end so engineers can design it, provision services on it, watch alarms on it, and break it on purpose, **without touching any physical hardware**. It is sold as an internal VCTI product and is the engineering team's flagship for the optical-management space.

2. What ONS is, in one page

ONS lets a user log in to a web application and interact with a complete optical network that lives entirely in software. The simulated network has the same vocabulary, the same constraints, and the same failure modes as a real one.

The product surface

Everything in ONS is reachable from one of **five tabs** on the left rail. The names are locked product nomenclature:

Tab	What it does	Status today
Topology	Live map of every element and every fibre link, on a real-world geographic basemap or a logical canvas.	Shipped

Elements	Searchable, filterable table of every network element (ROADM, OLA, TXP, MXP, Passive) with full create / edit / decommission / bulk import.	Shipped
Lightpaths	End-to-end optical services (Trails, Clients, NCs) provisioned through a six-step wizard backed by a path computation engine.	Planned
Alarms	ITU-T X.733 fault list with Live / Historical / Events sub-views and parent-child correlation.	Planned
Inventory	Physical hardware drill-down: Rack → Chassis → Shelf → Slot → Card → Port. Tree view and SVG faceplate view.	Shipped

Who uses it, and for what

Persona	Why they open ONS
Network operator (new hire)	Learn the product they will run in production, without risk to a live network.
QA engineer	Run repeatable scenarios — fibre cuts, card pulls, reroutes — against a known-good topology.
Software developer	Develop integrations against a stable, mockable backend that speaks real REST + WebSockets.
Architect	Validate designs at 500 elements / 10,000 light paths before committing capital.

v1 scope guardrail. ONS v1 is **state-only**. It models admin/operational state, port types, wavelength occupancy, and capacity. It does **not** model physical-layer impairments (OSNR, chromatic dispersion, PMD). Impairment modelling is a v2 candidate, layered in via GNPY.

3. Optical networking in plain English

If you have never worked on an optical network before, this analogy will carry you through every diagram in this document.

Optical concept	City-road analogy
Light (a specific colour / wavelength of laser light carrying data)	A stream of cars travelling between two places.
Fibre (a glass strand between two sites)	The road the cars drive on. One lane per fibre.
ROADM — Reconfigurable Optical Add-Drop Multiplexer	A freeway interchange. Decides which colour of light goes onto which road. Lets traffic get on or off here.
OLA — Optical Line Amplifier	A booster station on the side of the road. Makes the cars go louder so they reach the next interchange.
Transponder (TXP / MXP)	The translator. Paints the customer's grey Ethernet signal as a specific colour of light at the entrance, and unpaints it at the exit.

Light path	One end-to-end road trip: customer → transponder → ROADM → road → booster → road → ROADM → transponder → customer.
Alarm	An accident report. Carries a severity, what broke, whether traffic is interrupted, and a parent-child tree linking the original fibre cut to every road-trip it disrupted.

Three terms you will see all the time

- **DWDM grid** — the agreed set of "paint colours" available. ONS uses ITU-T G.694.1 50-GHz fixed grid: 96 channels in the C-band (191.30–196.05 THz).
- **Wavelength continuity** — a single light path must use the *same colour of light* end-to-end across every fibre it crosses. This is the single hardest constraint the path-computation engine has to satisfy.
- **X.733 severity** — the ITU-T standard for alarm severity: **Critical, Major, Minor, Warning, Indeterminate**. ONS uses these five and nothing else.

The seed network you will see on first login

When you start ONS for the first time, it builds a realistic regional metro backbone: **6 ROADMs** in a mesh, **2 ROADM spurs**, **4 in-line amplifiers**, **8 transponders**, **2 muxponders**. Wavelength utilisation sits around 25% — realistic for a day-1 network.

4. The five tabs — guided product tour

4.1 Sign-in

ONS uses username + password authentication. Passwords are hashed server-side with **Argon2id** at OWASP-2024 parameters; rate limiting and account lockout are enforced per (username, source IP) in Redis. The token, once issued, lives in an *HttpOnly Secure SameSite=Strict* cookie — never in `localStorage` .

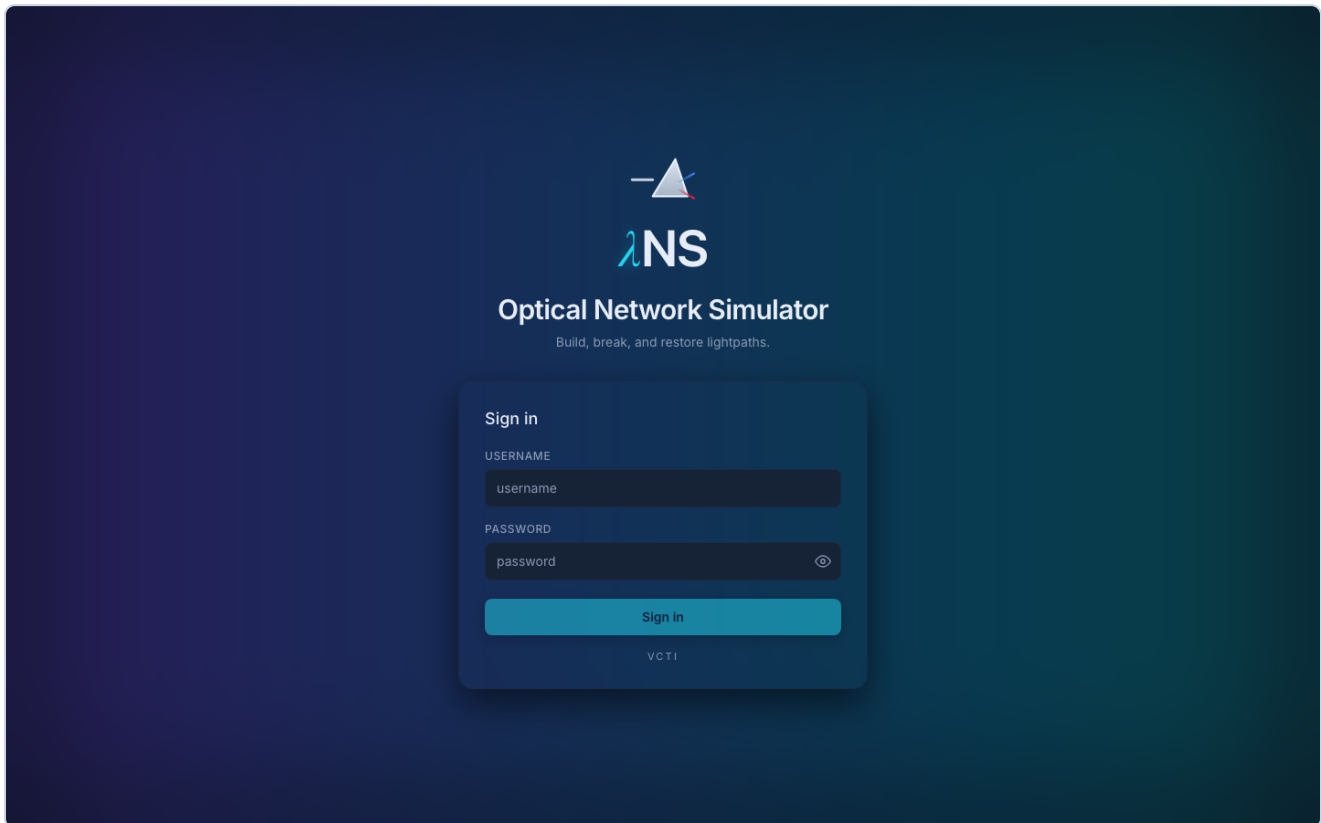


Figure 1 — The ONS sign-in screen. The brand mark and a one-line product promise sit above the form; nothing else competes for attention. A successful sign-in lands the user on the Topology tab.

4.2 Topology — geographic view

This is the screen every customer judges the product by. Each shape is a real network element placed at its actual latitude and longitude on a **MapLibre + Carto** basemap. The bottom-right attribution to Carto and OpenStreetMap is mandatory and unmovable — see ADR-0012.

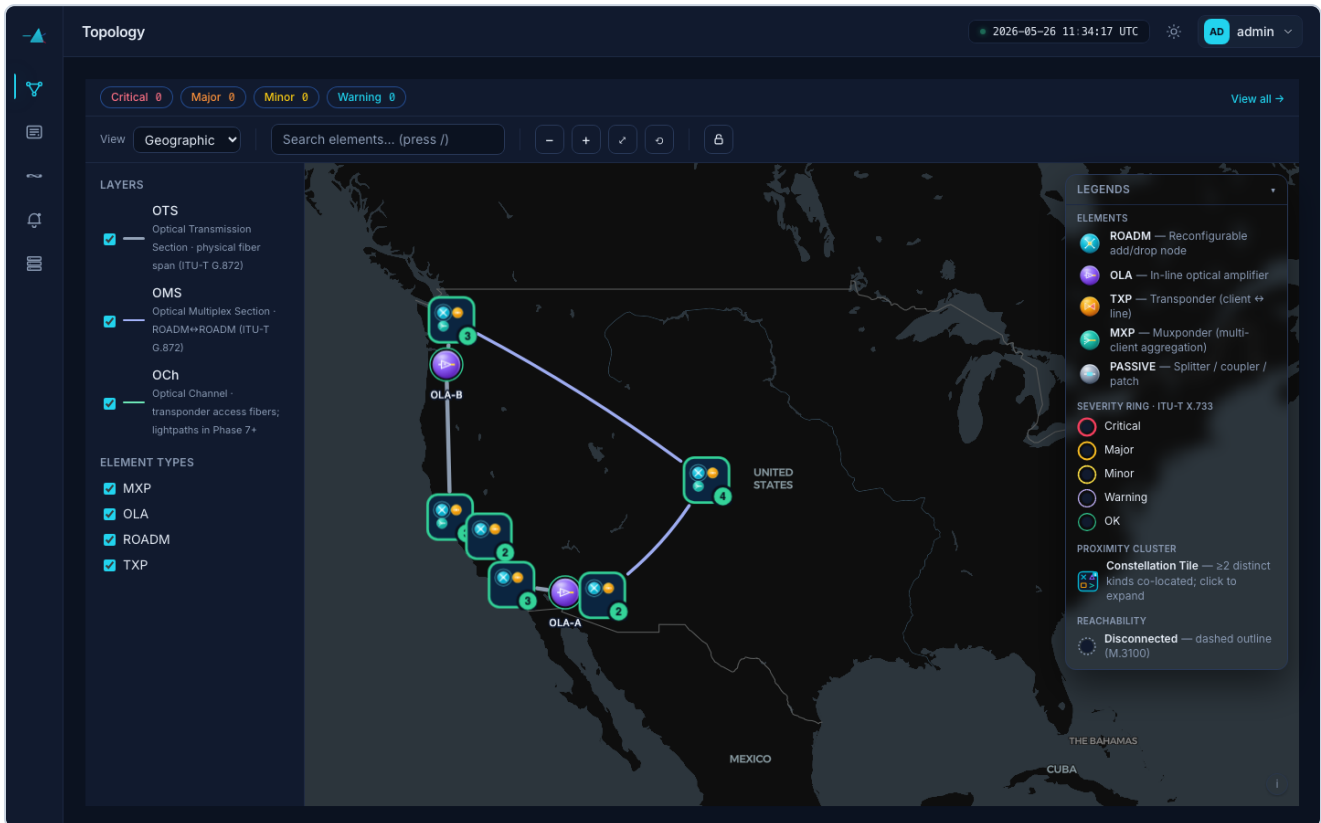


Figure 2 — Topology, geographic view. The left panel toggles three optical layers — OTS (physical fibre spans), OMS (ROADM-to-ROADM multiplex sections), OCh (transponder access fibres) — and the four element types. The right-hand legend decodes element shapes, severity rings (X.733), and the dashed-outline convention for disconnected elements (ITU-T M.3100). The top severity pills count Critical / Major / Minor / Warning across the visible viewport.

4.3 Topology — line-system view

The same network rendered as a logical graph, free of geographic distortion. This is the right tool when an operator cares about adjacency and degree count rather than physical distance — for example when tracing an OMS path from one ROADM to another.

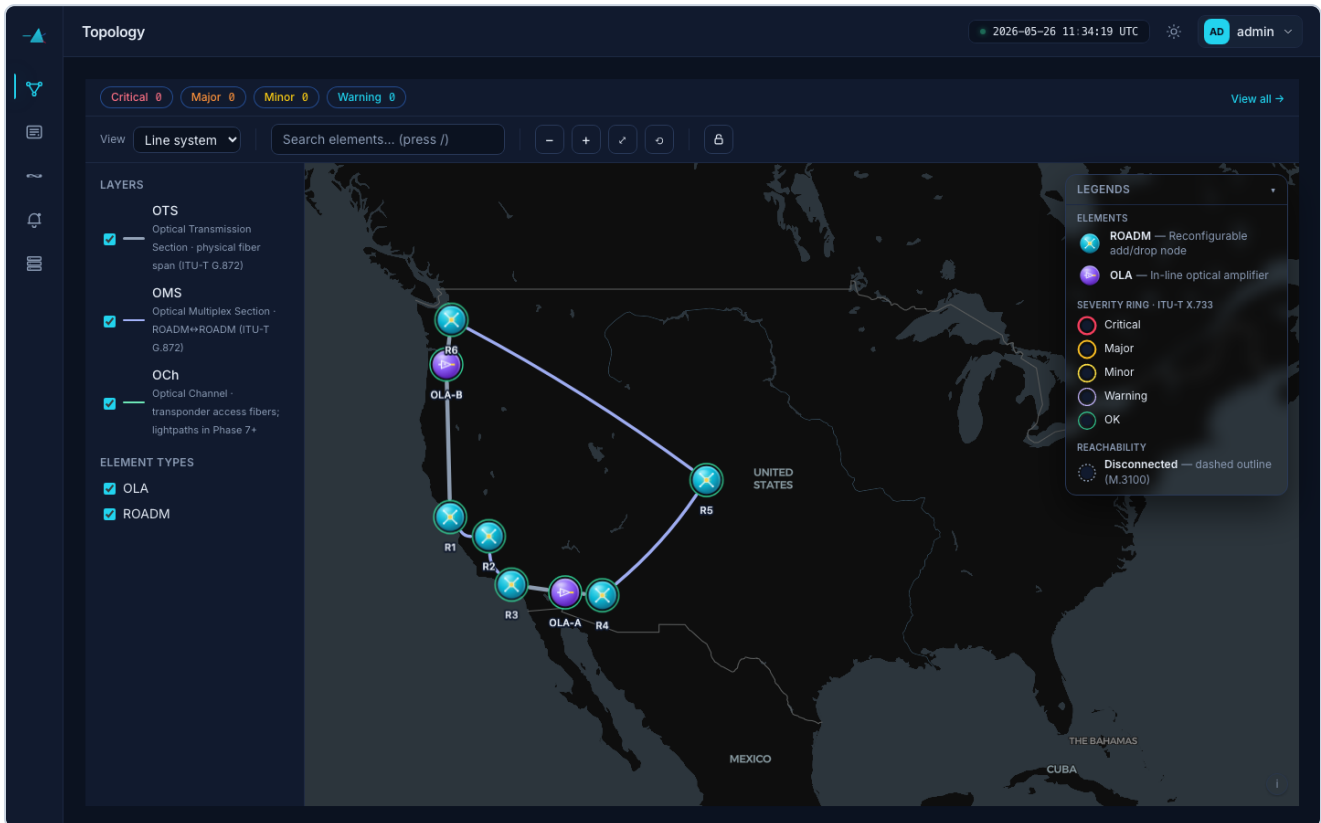


Figure 3 — Topology, line-system view. Elements keep the same shape and severity-ring semantics as the geographic view. Switching views preserves selection.

4.4 Elements

The Elements tab lists every managed network element in a high-density table. Each row has a deterministic icon per element type, a site name with coordinates, a connectivity state, the administrative state (ITU-T M.3100), the operational state, and a relative "last seen" timestamp.

Elements
Managed network elements with per-column filters, sort, export, create, edit, delete, bulk operations, and CSV / YAML / JSON import.

2026-05-26 12:02:17 UTC admin

+ Add Modify Remove Bulk... Import Export

Columns

Element	Site	Connectivity	Admin	Operational	Last seen
MXP-1 MXP · ONS-MXP-10×10-100G	San Francisco 37.800, -122.444	Connected	Unlocked	Enabled	3h ago
MXP-2 MXP · ONS-MXP-8×10-200G	Denver 39.764, -105.015	Connected	Unlocked	Enabled	3h ago
MXP-3 MXP · ONS-MXP-16×25-400G	Seattle 47.581, -122.357	Connected	Unlocked	Enabled	3h ago
OLA-A OLA · ONS-OLA-EDFA-RAMAN	Blythe 33.610, -114.596	Connected	Unlocked	Enabled	3h ago
OLA-B OLA · ONS-OLA-EDFA-RAMAN	Portland 45.515, -122.678	Connected	Unlocked	Enabled	3h ago
R1 ROADM · ONS-ROADM-MD8	San Francisco 37.775, -122.419	Connected	Unlocked	Enabled	3h ago
R2 ROADM · ONS-ROADM-MD8	Fresno 36.738, -119.787	Connected	Unlocked	Enabled	3h ago
R3 ROADM · ONS-ROADM-MD8	Los Angeles 34.052, -118.244	Connected	Unlocked	Enabled	3h ago
R4	Phoenix				

1-19 of 19

< Prev 1 / 1 Next >

Figure 4 — Elements table. Toolbar exposes Add, Modify, Remove, Bulk, Import, Export. Remove is destructive and always renders in the warning hue. The result-count chip in the top-right is the authoritative source of truth for total rows after filtering. The bulk import wizard is atomic by contract — either every row succeeds or none do.

4.5 Inventory — tree view

The Inventory tab drills into the physical hardware of any element. The **tree view** renders the full ITU-T M.3100 lineage — Rack → Chassis → Shelf → Slot → Card — as one keyboard-navigable tree. Each row carries a leading severity dot (X.733) so an operator can find trouble without opening every branch.

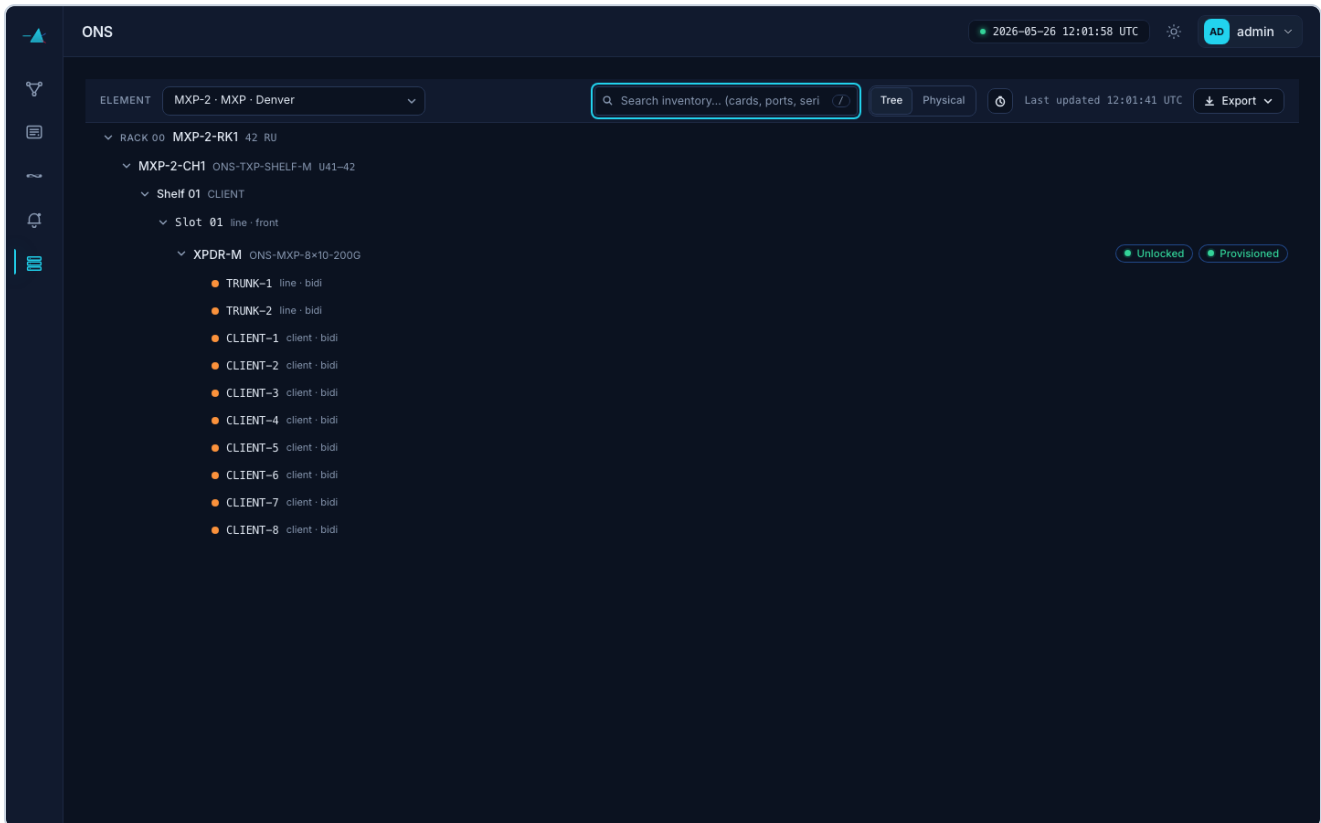


Figure 5 — Inventory tree view. The element selector at the top scopes the tree to one element at a time. The active row receives the accent-coloured indicator rail on the left. Selection is preserved across expand and collapse. JSON and CSV export are streamed (bounded memory regardless of result size).

4.6 Inventory — physical view (faceplate)

The **physical view** is a schematic, non-photographic rendering of the selected element's rack. Chassis stack top-to-bottom within each rack at their assigned rack-unit (U) positions; each chassis declares its severity rollup chip. Card faceplates draw the port grid; each port carries an X.733-coloured LED for live severity.

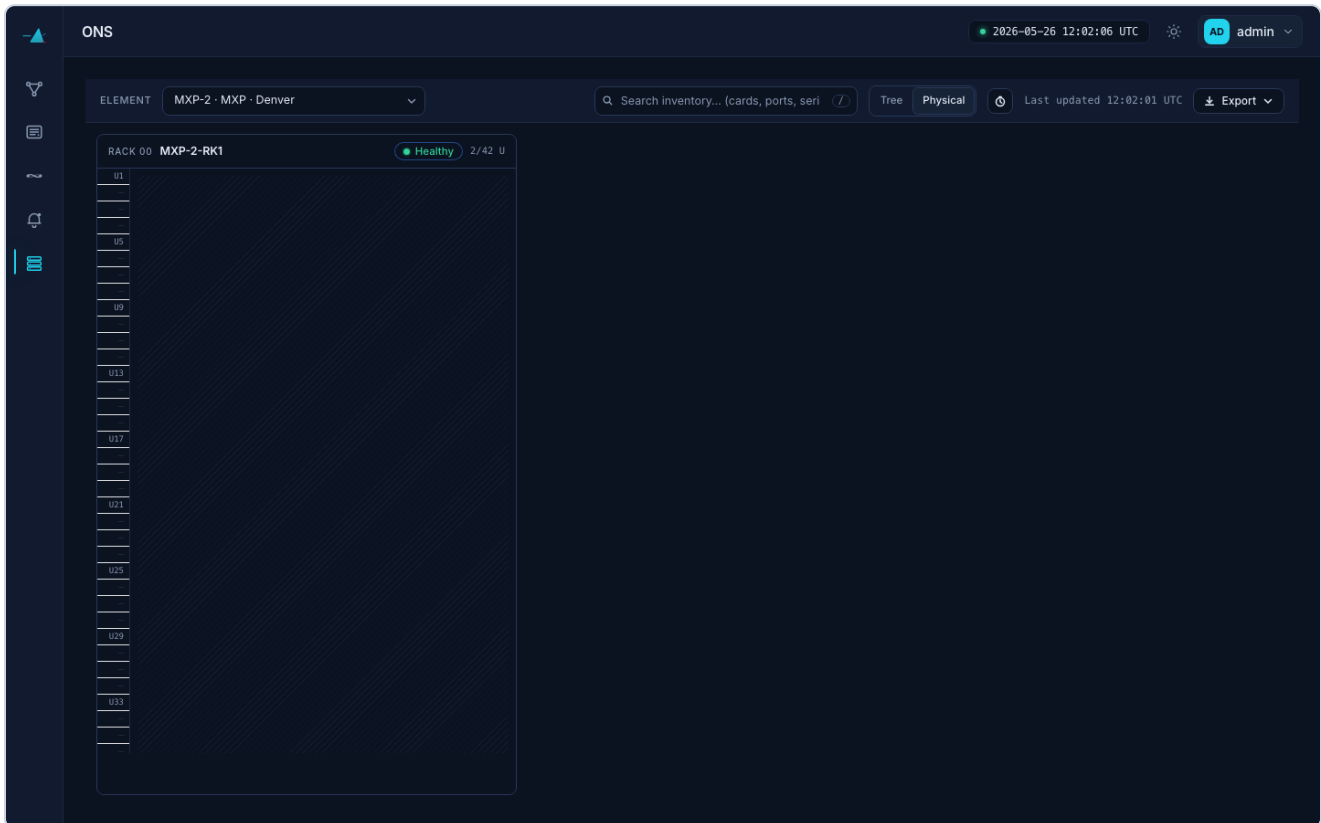
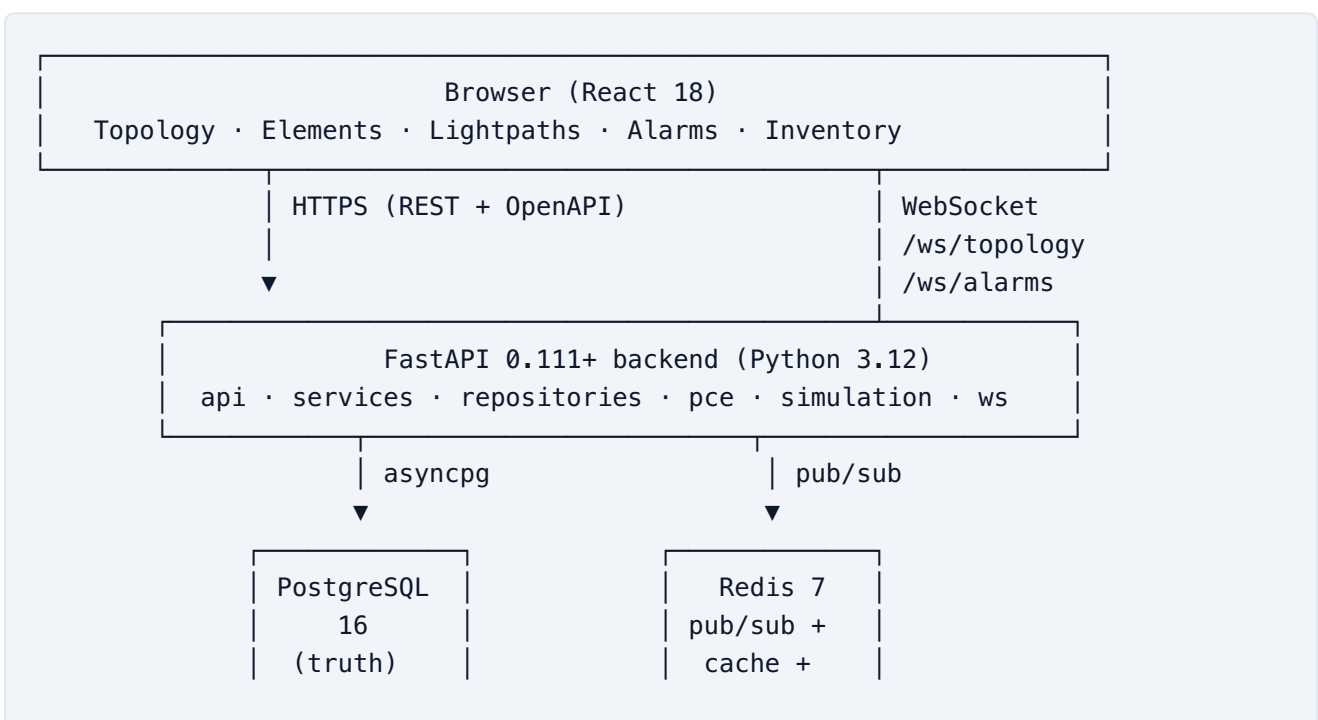


Figure 6 — Inventory physical view. The view is a 2-D keyboard grid: arrow keys move between ports, Enter opens the resource drawer. Originality is non-negotiable here: shapes are schematic, with no photo-realistic hardware imagery and no copied faceplate layouts from any external product.

5. System architecture at a glance

ONS is a conventional three-tier web application with one extra realtime channel. Reads and mutations go over HTTPS; live updates (alarms, topology deltas) ride a WebSocket; long-running work (PCE, alarm correlation) runs as background tasks inside the backend process.



The five backend modules to learn first

Module	What it owns
api/	FastAPI routers, one file per resource. No business logic here.
services/	Business logic. Mutations always go through a service; routers never call repositories directly.
repositories/	Database access. Pure ORM. Returns models, never schemas.
pce/	Path Computation Engine. Pure-function graph algorithms over NetworkX. Fully unit-testable without a database.
ws/	WebSocket managers. Receive Redis pub/sub events and fan them out to connected clients.

6. Technology stack, by layer

Concern	Choice	Why
Backend language	Python 3.12+	Mature async story; SQLAlchemy 2.x typed Mapped[]; matches the team's depth.
HTTP framework	FastAPI 0.111+	OpenAPI auto-generation drives the frontend's TypeScript types.
ORM + migrations	SQLAlchemy 2.x + Alembic	Typed models; Alembic from day one — never edit applied migrations.
Validation	Pydantic v2	Boundary schemas: Create / Update / Read / ReadDetailed per resource.
Database	PostgreSQL 16	Native enums (M.3100, X.733), tsvector + GIN for search, partial unique indexes for alarm dedup.
Realtime + cache	Redis 7	Pub/sub backplane for WebSocket fan-out; sliding-window rate limits; sessions.
Frontend	React 18 + TypeScript strict	<code>exactOptionalPropertyTypes</code> on; OpenAPI-generated types only.
Build	Vite	Fast HMR; native ESM.
UI primitives	Tailwind tokens + Zustand + TanStack Query	Token-only palette, never raw hex; client state in Zustand, server state in TanStack Query.
Graph rendering	Cytoscape.js (line-system view) + MapLibre GL (geographic view)	Cytoscape for free-form graph layout; MapLibre + Carto raster tiles for the basemap.
Path computation	NetworkX 3.x	Dijkstra + Yen's k-shortest-path; wavelength-continuity solver layered on top.

Auth	Argon2id + JWT in HttpOnly cookie	OWASP-2024 parameters; cookie not Authorization header — protects against XSS exfiltration.
Container	Docker Compose (dev) + Helm chart (prod)	One-command local dev; Kubernetes-ready for enterprise on-prem delivery.
Observability	structlog · Prometheus · OpenTelemetry · Grafana	JSON logs with request-id; RED metrics per endpoint.

7. How to run it on your laptop

From a clean clone of the repository, with Docker Desktop running:

```
# 1. Start Postgres + Redis (Docker Compose)
./scripts/dev-up.sh

# 2. Backend — Python venv, migrations, seed, server
cd backend
uv sync                                # or: python -m venv .venv && pip install -e .
source .venv/bin/activate
alembic upgrade head
python -m simulator_seed                # builds the seed network
python -m simulator_seed bootstrap-admin --email you@vcti.com
BACKEND_ENV=local uvicorn app.main:app --port 8800 --host 127.0.0.1

# 3. Frontend — pnpm install, dev server
cd ../frontend
pnpm install
pnpm dev
# opens at http://127.0.0.1:5273

# 4. Sign in with the admin account you just created.
```

Things to know on day one

- **Banned-terms scanner** — every documentation merge is gated by `scripts/check-banned-terms.sh`. Never name a commercial product anywhere in this repository.
- **Migrations are immutable** — once an Alembic revision is in `main`, you create a new revision; you never edit the old one.
- **OpenAPI is the contract** — the frontend's TypeScript types are generated from the backend's OpenAPI schema. Hand-written types are forbidden.
- **Tokens, not hex** — Tailwind tokens only on the frontend. Themes swap by changing CSS custom properties, never by editing components.
- **Reduced motion is honoured** — every animation collapses to near-zero when the user prefers reduced motion (WCAG 2.3.3).

8. Where to go next

This is a ramp-up document, not the manual. When you finish it, the next reads (in order) are:

Topic	Doc
Full product introduction	<code>docs/00-introduction/what-is-ons.md</code>
Optical concepts, depth	<code>docs/00-introduction/optical-networks-101.md</code>
Guided tour of the five tabs	<code>docs/00-introduction/the-five-tabs.md</code>
Glossary of every term	<code>docs/00-introduction/glossary.md</code>
Architecture deep-dive	<code>docs/architecture/system-overview.md</code>
Backend architecture	<code>docs/architecture/backend-architecture.md</code>
Frontend architecture	<code>docs/architecture/frontend-architecture.md</code>
Data model (ER diagram + every table)	<code>docs/architecture/data-model.md</code>
What ships today vs. what is planned	<code>docs/architecture/implementation-status.md</code>
Architecture Decision Records	<code>docs/adr/</code> (0001 → 0013)
The complete plan	<code>plan-opticalNetworkSimulator.prompt.md</code> (project root)

One sentence to take with you

ONS exists so the engineers who will one day operate a real optical network can practise on a software one — and so the engineers who build the operating tools can develop them against a simulator that behaves like the real thing.